

i.MX6U 嵌入式 Linux 驱动开发指南

21 天（三周）学习计划 · 面向「嵌入式 Linux 网关 + MCU 固件」岗位定制
对象：正点原子 I.MX6U ALPHA/Mini 开发板教程 V2.0.1（全书 1963 页，76 章 + 附录）
生成日期：2026-06-06

一句话核心：这本书 1963 页，21 天不可能逐页啃完；但你那份 JD 也根本不需要全啃。本计划的灵魂是——**砍裸机、轻啃移植、重压驱动**。把约 80% 的时间压到第四篇「Linux 驱动开发」，因为那才是 JD 逐条要的东西（设备树、字符设备驱动、UART/SPI/I2C/GPIO 调通、无线模组驱动移植）。

一、核心策略与全书骨架

我通读全书目录与四篇分篇后，把整本书拆成下面四块，并标出各自与你 JD 的关系：

篇	起始页	章节	与你 JD 的关系
第一篇 Ubuntu 入门	p47	1-3	你已会，速通
第二篇 裸机开发	p155	4-29	战略性跳过（除环境章 Ch4）
第三篇 系统移植	p695	30-39	轻啃但必做 —— 驱动的前置拦路虎
第四篇 Linux 驱动开发	p1053	40-76	★★★ 全书重点，80% 时间在这里
附录	—	A1-3 / B1-4 / C1	B3/B4 调试要做，其余按需

为什么敢砍第二篇（裸机 540 页）？

- ① 你已有 STM32F407 裸机底子：i.MX6U 的寄存器级点灯/中断/定时器对你重复劳动。
 - ② JD 不需要：这份 JD 全程要的是 Linux 应用层 + 驱动，没有任何写 i.MX6U 裸机寄存器的需求。
 - ③ 会重讲：裸机里的 I2C/SPI/串口，第四篇会用 Linux 子系统重新讲一遍，直接学子系统版即可。
- 标注规则：★★★ = JD 直接命中，必须吃透；★★ = 驱动必备；★ = 快速过 / 选学。

二、第一周（Day 1-7）：搭环境 + 轻啃移植，目标「把板子跑起来」

本周唯一目的：拥有一个能跑你自己驱动的 Linux 板子 + 一套能编译能调试的环境。这是第二、三周一切实验的地基。

Day 1 — 第一篇速通 + 环境搭建（上）

- 第 1-3 章快速翻，只精读 Ch3 Makefile 语法（p141-154），驱动开发天天写 Makefile。
- ★★★ Ch4 开发环境搭建（p156-222）：交叉编译工具链(4.3)、NFS/SSH 开启(4.2)、VSCode(4.5)。

交付 arm-linux-gnueabi-hf-gcc 能编译 hello、NFS 能挂载（4.3.3 验证通过）。

Day 2 — 平台认知 + 裸机「战略性略过」

- ★ Ch5 平台介绍 (p223–287)：只看你要用的部分原理图 (LED/按键/串口/I2C/网口/EMMC)。
- ★ Ch6 架构 (p288) 略读；Ch8 汇编 LED (p304) / Ch10 C 语言 LED (p344) 只看「IO 复用→IO 配置→GPIO 输出」这条寄存器路径。
- 明确决定：Ch7、9、11–29 裸机例程本轮跳过，驱动篇遇到对应外设再回查。

交付 能口述 i.MX6U 一个 GPIO 从复用到输出的寄存器流程。

Day 3 — U-Boot 使用 + 移植 (上)

- ★★ Ch30 U-Boot 使用 (p696–739)：重点 tftp/nfs/fatload、bootcmd/bootargs —— 这些命令天天用。
- ★★★ Ch33 U-Boot 移植 (p859) 开始：移植 NXP 官方 uboot 到正点原子板。
- Ch31/32 顶层 Makefile、启动流程「详解」本周只略读，先会用再深究。

交付 自编 u-boot.imx 能启动、网络 ping 通、tftp 能下载。

Day 4 — U-Boot 移植 (下)

- Ch33 收尾 + ★ Ch34 图形化配置 menuconfig (p911–928) 快速过。

交付 U-Boot 完整移植通过，能从网络 / EMMC 引导。

Day 5 — Linux 内核移植

- ★★★ Ch37 内核移植 (p973–1008)：编译 NXP 内核、改设备树、使能网络/LCD、menuconfig。
- Ch35/36 内核 Makefile、启动流程略读。

交付 自编 zImage + .dtb 能启动内核。

Day 6 — 根文件系统构建

- ★★★ Ch38 根文件系统 (p1009–1033)：busybox 编译、构建 rootfs、库与 /etc、NFS 挂载 rootfs。

交付 板子用 NFS 挂载你构建的 rootfs，启动到 shell 命令行。

Day 7 — 系统烧写 + 调试环境 + 第一周复盘

- ★★★ Ch39 系统烧写 (p1034–1052)：MfgTool 把三件套烧进 EMMC，脱机独立启动。
- ★★ 附录 B3/B4：嵌入式 GDB + VSCode+gdbserver 图形化调试 (后面写驱动靠它)。

交付 板子上电独立启动到你的系统；gdbserver 远程调试 hello world 通。

防坑 移植最容易卡时间。若 Day3–6 哪一步卡死超过半天，先烧正点原子出厂系统镜像保证板子能跑，让第二、三周的驱动实验不被阻塞，移植回头再补。别让「拦路虎」拖垮整个 21 天。

三、第二周 (Day 8–14)：Linux 驱动核心框架 —— 全书重中之重

本周是 JD 的心脏：「字符设备驱动 + 设备树 + 驱动分离框架」。每一章都用同一个学法：抄一遍 → insmod 跑通 → 关书默写 → 改成新功能。

Day 8 — 字符设备驱动 + LED 驱动（驱动界的 Hello World）

- ★★★ Ch40

字符设备驱动（p1054–1080）：模块加载卸载、注册设备号、file_operations、mknod。必须吃透。

- ★★★ Ch41 LED 驱动（p1081）：第一个真正碰硬件的驱动（ioremap + open/read/write）。

交付 led.ko + app 通过 write 控制 LED。

Day 9 — 新字符设备驱动（现代写法，以后所有驱动的模板）

- ★★★ Ch42 新字符设备驱动（p1094）：自动分配设备号、cdev、自动创建 /dev 节点、私有数据。

交付 用新框架重写 LED，自动生成 /dev/led。

Day 10 — Linux 设备树（JD 白纸黑字要求，重点中的重点）

- ★★★ Ch43 设备树（p1109–1149）：DTS 语法、节点/属性、of 操作函数、/proc/device-tree。

交付 自己在设备树里加一个节点描述 LED 硬件，能在 /proc/device-tree 看到。

Day 11 — 设备树下的驱动 + pinctrl/gpio 子系统

- ★★★ Ch44 设备树下 LED（p1150）：把硬件信息从代码搬进设备树，of_* 读属性。

- ★★★ Ch45 pinctrl + gpio 子系统（p1162）：gpio_request/direction/set —— JD「GPIO 调通」的规范方式。

交付 用 pinctrl + gpio 子系统重写 LED（不再裸操作寄存器），这才是规范写法。

Day 12 — 并发竞争 + 内核定时器

- ★★ Ch47 并发竞争（p1205） + Ch48 实验（p1219）：原子操作、自旋锁、信号量、互斥体。

- ★ Ch49 按键（p1243）快速 + ★★ Ch50 内核定时器（p1254）。

交付 给 LED 驱动加原子变量，做到「同一时刻只允许一个 app 打开」。

Day 13 — 中断 + 阻塞 / 非阻塞 IO

- ★★★ Ch51 中断（p1271）：request_irq、上半部/下半部、tasklet、工作队列。

- ★★ Ch52 阻塞 / 非阻塞 IO（p1295）：等待队列、poll（select/poll/epoll 底层）。

交付 按键中断驱动，app 用阻塞 read 等按键、用 poll 监听。

Day 14 — 异步通知 + platform 框架（驱动分离的灵魂） + 复盘

- ★★ Ch53 异步通知（p1321）：SIGIO、fasync。

- ★★★ Ch54 platform 设备驱动（p1334） + Ch55 设备树下 platform 驱动（p1361）：总线/设备/驱动模型、驱动分离分层 —— Linux 驱动工程化的核心思想，JD「系统集成」的底座。

交付 用 platform 框架 + 设备树重构 LED（probe/remove），真正理解「设备与驱动分离」。

四、第三周（Day 15–21）：外设 / 总线 / 网络驱动 —— 逐条命中 JD

JD 原话：「UART、SPI、I2C、GPIO 接口调通」「无线模组驱动移植与适配」。这一周每一天几乎对应 JD 的一行。

Day 15 — MISC + INPUT 子系统

- ★ Ch56 自带 LED 驱动 (p1371) 速读 + ★★ Ch57 MISC 驱动 (p1380) (常用杂项设备)。
- ★★ Ch58 INPUT 子系统 (p1391) : input_dev 上报按键/坐标事件, evtest 验证。

交付 MISC 框架写一个驱动; input 子系统上报按键, evtest 看到事件。

Day 16 — Linux I2C 驱动 (JD「I2C 调通」+ 直接升级你的传感器节点)

- ★★★ Ch61 I2C 驱动 (p1448) : i2c 总线/设备/驱动、i2c_transfer、设备树配从设备。
- 可顺带 ★ Ch60 RTC 驱动 (p1435) 当 I2C 设备应用速览。

交付 在 i.MX6U 上用 Linux I2C 框架驱动 SHT30, 读到温湿度。把你作品的传感器从「WS63 裸机软件 I2C」升级成「Linux 驱动」, 正好补 JD 缺口。

Day 17 — Linux SPI 驱动 (JD「SPI 调通」)

- ★★★ Ch62 SPI 驱动 (p1482) : spi_device/driver、spi_transfer、设备树配 SPI。

交付 用 Linux SPI 框架驱动一个 SPI 器件 (如板载 ICM20608 六轴), 读到数据。

Day 18 — Linux 串口 / UART 驱动 (JD「UART 调通」+ 对接近场模组)

- ★★★ Ch63 RS232/485/GPS 驱动 (p1512) : Linux 串口框架、termios、把外部模组接进 Linux 串口。

交付 i.MX6U 通过串口与你的 STM32F407 / WS63 通信, 收到终端数据。这就是你作品「近场终端 → 网关」那一环在 Linux 侧的落地。

Day 19 — Linux 网络驱动 (三大类驱动之一)

- ★★★ Ch69 网络驱动 (p1684, 约 70 页较大) : MAC/PHY、net_device、sk_buff、收发流程。重点理解框架与移植, 不必死抠每行。

交付 有线网卡驱动通, ifconfig 配 IP、ping 通 PC。

Day 20 — Linux WIFI 驱动移植 (★ JD「无线模组驱动移植」最直接命中 ★)

- ★★★ Ch70 WIFI 驱动 (p1753) : USB/SDIO WIFI 模组驱动移植、加载固件、wpa_supplicant 联网。
- 有空瞄一眼 ★ Ch71 4G 通信 (p1773) : PPP 拨号, 网关备用上云链路。

交付 WIFI 模组在 i.MX6U 上移植成功并连上路由器。这几乎就是 JD「无线模组驱动移植与适配」的标准答案, 简历可直接写。

Day 21 — 收尾：把驱动能力焊进作品 + 总复盘（不学新章节）

- 整合规划你的「智能家居网关」在 i.MX6U 上的落地，逐条对照 JD 打勾：
- — 自写字符 / platform 驱动（传感器 / GPIO 控继电器）← Ch40–55
- — 设备树改造 ← Ch43–45
- — BLE/Zigbee 模组走 Linux 串口 / 驱动移植 ← Ch63 + Ch70 思路
- — 网关主程序：串口/I2C 收数 → MQTT 上云 → 弱网重连状态机（应用层，用你已有 MQTT 经验，书外）
- — OTA（断电保护 / 回滚）← 列入后续 TODO
- 选学补充（有余力再看）：★ Ch74 Regmap API（p1815）、Ch66 CAN（p1610）、Ch68 块设备（p1661）。

交付 一份《i.MX6U 网关作品改造方案 + 已完成驱动清单》，对照 JD 逐条打勾，更新简历技能点。

五、明确「本轮不看」的章节（及理由）

心里有数，不因「跳过」而焦虑：

章节	处理	理由
Ch7、9、11–29 裸机外设	跳	你有 STM32 底子；子系统会重讲；JD 不写 i.MX6U 裸机
Ch59 LCD / Ch64 触摸屏 / Ch72 RGB转HDMI	略读或跳	JD 不重图形/GUI；你这是网关岗，不是 HMI 岗
Ch65 音频驱动	跳	ALSA/ASoC 又大又偏，与 JD 无关
Ch67 USB / Ch68 块设备	缓学	篇幅大，非 JD 核心（块设备了解概念即可）
Ch73 PWM / Ch75 IIO / Ch76 ADC	选学	小且边缘，有空当扩展

六、执行层面的提醒

- **配视频一起学**：驱动篇强烈建议正点原子「原子哥」配套视频 + 文档对照；配套例程源码在开发板光盘，先跑通再改。
- **每章学法固定四步**：抄一遍 → insmod 跑通 → 关书默写 → 改成新功能。能默写出 file_operations / probe 框架，才算真懂。
- **节奏假设**：本计划按每天约 6–8 小时（应届全力备战）排。每天只有 3–4 小时，就当 6 周计划，每天拆一半 —— 尤其第一周移植和第三周网络/WIFI 容易超时。
- **简历落点**：真正能写进简历、面试官认的是 Ch42/43/45（字符+设备树+gpio子系统）、Ch54/55（platform）、Ch61/62/63（I2C/SPI/串口）、Ch70（WIFI 模组移植）—— 逐条对应你 JD 那五条职责。

下一步：需要的话，我可以把任意一天再拆成「单步指令」（你做完一步、贴结果给我，我再给下一步）。第一周的 U-Boot / 内核移植最容易卡，可从 Day 3 第一步开始带你单步走。